

Tokenization and Weighting

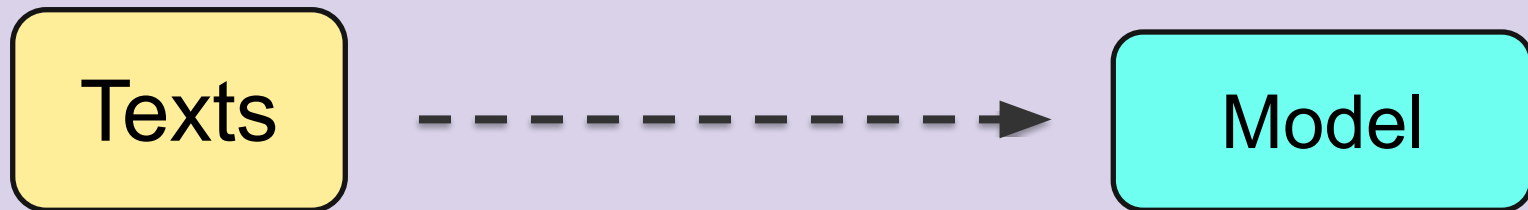
Klara Venglarova



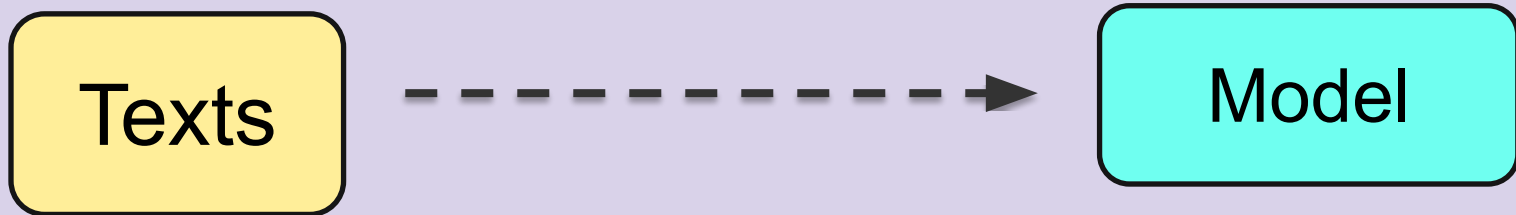
Outline

1. Big Picture: Where are we and why?
2. Text Pre-Processing
3. Tokens and Tokenization
4. Frequencies
5. Bag-of-Words Representation and CountVectorizer
6. TF-IDF and c-TF-IDF

Where are we and Why?



- BUT: Machine-learning models perform mathematical operations on numerical representations, so we need to transform our text into numbers
- This involves several steps: (next slide)



- **Pre-processing** and **tokenization** (tokenize = turn the continuous text into smaller units such as words or subwords)
- **Vectorization** = turning these units into numbers (vector = „list of numbers“)
- Vectors as **inputs** into the model

Text Pre-Processing in NLP

Pre-processing is the set of steps applied to raw text to

- **clean,**
- **normalize,** and
- **standardize**

it so that it can be effectively processed by NLP models.

Text Pre-Processing in NLP: Why?

- Raw text is messy: punctuation, capitalization, typos, numbers, stopwords, emojis.
- Pre-processing helps:
 - **Reduce noise** → fewer meaningless tokens
 - **Normalize text** → make similar words consistent (run vs. running)
 - **Reduce dimensionality** → smaller, cleaner vocabulary

I am running to a school.

Common Pre-Processing Steps

- **Lowercasing:** Convert all text to lowercase
 - e.g.: “Galaxy” → “galaxy”
- **Punctuation removal:** Remove symbols, commas,...
 - e.g.: “far, far away...” → “far far away”
- **Stopwords removal:** Remove common words that add little meaning
 - e.g.: “in a galaxy” → “galaxy”

A long time ago in a Galaxy far, far away...

Common Pre-processing Steps II.

- **Tokenization:** Split text into small units (e.g. words/subwords)
 - e.g.: “long time ago” → [“long”, “time”, “ago”]
- **Lemmatization:** Reduce words to their base form
 - e.g.: “running” → “run”
- **Stemming:** Reduce words to their root form
 - “better” → “bet”
- **POS tagging:** Adding information about Part-of-the-speech
 - e.g.: [“long”, “time”] → [“long_ADJ”, “time_NOUN”]

Tokenization

- **Token** = the smallest unit of text, such as a word, subword or punctuation mark
- **Tokenization** = the process of breaking text into these units

„a smart dog“ >> „a“, „smart“, „dog“

- Token =? Word
- Matter of definition: Henry V. = one word or two?
 - U.S.A.
 - E-Mail
 - Rock ‘n’ roll
 - New York

Types and Tokens

- **Token** = every single unit in a text, individual instances
- **Type** = a dictionary of all words used in a text (all unique words in a text)
- Example (if token = words):

„a smart dog is using a smart phone“

- Tokens: a, smart, dog, is, using, a, smart, phone
- Types: a, smart, dog, is, using, phone

Exercise

„a long time ago in a galaxy far, far away“

- if token == word and we ignore interpunction:
 - How many tokens?
 - How many types?
-
- Tokens: a, long, time, ago, in, a, galaxy, far, far, away {10}
 - Types: a, long, time, ago, in, galaxy, far, away {8}



Source:
[https://en.wikipedia.org/wiki/Galaxy#/media/File:NGC_4414_\(NASA-med\).jpg](https://en.wikipedia.org/wiki/Galaxy#/media/File:NGC_4414_(NASA-med).jpg)

Tokenization

- other units than words
- In BERT – subword tokenization using WordPiece
- Example of the principle: *“unhappiness”* → [*“un”, “##happy”, “##ness”*]
- *“translatability”* → [*“translate”, “##ability”*]
- Out-of-vocabulary words („unforgetability“)
- more information here:
<https://huggingface.co/learn/llm-course/chapter6/6>

Frequencies

- frequency of a type = how many times it appears in the document

„a long time ago in a galaxy far, far away“

- frequency of *far* = 2
- frequency of *galaxy* = 1

Bag-of-Words Representation (BoW)

- Bag-of-Words (BoW) is a **text representation** method that converts a document into a **vector** based on the **frequency** (or presence) of words, ignoring grammar and word order.
- Turns the document into a matrix of types and their frequencies.
- Turning text into a BoW vector = **feature extraction** or **vectorization**.

Bag-of-Words

```
Corpus = ["dog cat dog",  
          "cat bird"]
```

features

Document	bird	cat	dog
1	0	1	2
2	1	1	0

BoW Creation Steps I.

A long time ago in a galaxy far, far away...

1. **Tokenization** (+ lowercase)

[a, long, time, ago, in, a, galaxy, far, far, away]

2. **Build Vocabulary** (from types = unique words)

{a, long, time, ago, in, galaxy, far, away}

BoW Creation Steps II.

3. Count Word Frequencies

[a, long, time, ago, in, a, galaxy, far, far, away]

Word Count

a 2

long 1

time 1

ago 1

in 1

galaxy 1

far 2

away 1

BoW Creation Steps II.

4. BoW Vector

If we order the vocabulary as:

[a, long, time, ago, in, galaxy, far, away]

Then the BoW vector is:

[2, 1, 1, 1, 1, 1, 2, 1]

BoW Creation Steps IV.

- differently written as:

a longtimeago in galaxy far away

2 1 1 1 1 1 2 1

BoW Creation

- We want to add a second sentence: ***A galaxy is far away.***

	a	long	time	ago	in	galaxy	far	away	is
sentence 1	2	1	1	1	1	1	2	1	0
sentence 2	1	0	0	0	0	1	1	1	1

CountVectorizer

In Python, CountVectorizer is a tool that turns a collection of text documents into a Bag-of-Words representation, i.e. matrix of token counts, showing how many times each word appears in each document.

CountVectorizer and its Parameters

1. **binary** → Frequency vs. Binary BoW

- **Default (**binary=False**)** → Counts word frequencies.
 - "far" appears twice → value = 2.
- **Binary (**binary=True**)** → Marks only presence (1) or absence (0).
 - "far" appears twice = is present → value = 1.

👉 Use case: when **repetition doesn't matter** (e.g., spam detection — presence of “buy”, “free”, “offer” is enough).

N-grams

Unigrams: [a, long, time, ago, in, a, galaxy, far, far, away]

Bigrams:

a long, long time, time ago, ago in, in a, a galaxy, galaxy far, far far, far away

Trigrams:

a long time, long time ago, time ago in, ago in a, in a galaxy, a galaxy far, galaxy far far, far far away

4-grams:

a long time ago, long time ago in, time ago in a, ago in a galaxy, in a galaxy far, a galaxy far far, galaxy far far away

N-grams: Exercise

The solar system has eight planets.

a) Bi-grams:

b) Tri-grams:

[„The solar“, „solar system“, „system has“, „has eight“, „eight planets“]

[„The solar system“, „solar system has“, „system has eight“, „has eight planets“]

CountVectorizer: ngram_range

2. ngram_range

- **Default:** (1, 1) → unigrams only (single words).
- (1, 2) → unigrams **and** bigrams.
 - Example: "far away" becomes its own token as well as "far" and "away".
- (2, 2) → only bigrams.

👉 N-grams capture **phrases** and order — helps when word combinations carry meaning.

N-Grams

- Why using n-grams?
- *Henry V.*
- *New York*
- *far away*
- *not happy*

CountVectorizer: max_features

3. max_features

- Restricts vocabulary to the top k most frequent words.
- Example: if `max_features=5`, only the 5 most common words across the corpus are kept.

👉 Useful for very large corpora where the full vocabulary would explode in size.

CountVectorizer: min_df and max_df

4. min_df and max_df

- **min_df** = minimum number (or proportion) of documents a word must appear in to be included.
 - Example: **min_df=2** → ignore words that appear in only 1 document.
- **max_df** = maximum number (or proportion) of documents a word can appear in.
 - Example: **max_df=0.9** → ignore words that appear in more than 90% of documents (often stopwords).

👉 These help clean the vocabulary:

- Drop **rare words** (too specific, not generalizable).
- Drop **very common words** (uninformative, e.g., “the”, “is”).

CountVectorizer: Stop Words

5. `stop_words`

- Removes common words (like “the”, “in”, “a”).
- Built-in: "`english`", or a custom list.
- prepositions, articles, pronouns...

👉 Prevents filler words from dominating the BoW.

Stop Words: Exercise

- Which words would you consider stop words?

Anyway, the weather will be nice.

Anyway, the weather will be nice.



Source:

<https://cdn-icons-png.flaticon.com/512/10127/10127236.png>

CountVectorizer: lowercase

6. lowercase

- Default = `True`.
- Converts all text to lowercase → “Galaxy” = “galaxy”.
- (opposite: UPPERCASE)

CountVectorizer: token_pattern

7. token_pattern

- Regex that defines what counts as a “word.”
- Default: tokens of 2 or more alphanumeric characters ('(?u)\b\w\w+\b').
- Can be customized to include single letters (like “a” or “I”), or handle special symbols.

CountVectorizer in Python

```
from sklearn.feature_extraction.text import CountVectorizer  
  
import pandas as pd  
  
docs = [  
    "A long time ago in a galaxy far, far away.",  
    "The galaxy is far away."  
]
```

CountVectorizer in Python

```
# Custom CountVectorizer
```

```
vectorizer = CountVectorizer(
```

```
    lowercase=True,          # convert all text to lowercase
```

```
    stop_words='english',    # remove English stopwords like "the", "in"
```

```
    binary=True,             # binary bag-of-words (presence/absence)
```

```
    ngram_range=(2,2),       # include only bigrams
```

```
    min_df=1,                # token must appear in at least 1 document
```

```
    max_df=0.9               # ignore tokens appearing in >90% of docs
```

```
)
```

```
# Fit and transform - fit to train the features dictionary and then transform to transform text into a vector:
```

```
X = vectorizer.fit_transform(docs)
```

CountVectorizer in Python

```
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())  
print(df)
```

	ago	galaxy	far	far	long	time	time	ago
0		1		1		1		1
1		0		0		0		0

BoW Advantages

- an easy representation to start with
- simplicity - to understand, explain, and implement
- implementation in Python libraries
- allows for certain flexibility (n-grams, lemmatization, stopwords...)
- effective baseline - often performs surprisingly well for tasks like spam detection, sentiment analysis, or document classification
- interpretability - features (words) are human-readable, so you can see which words drive predictions

Problems with BoW

- BoW loses words order
- BoW misses context
- BoW treats all words equally
- Common words might dominate just because they appear often, not because they're informative.
- Example: in a large corpus, words like “the” or “said” would drown out more meaningful words.
- High dimensionality
- Extreme matrix sparsity (most of the matrix are actually ,0‘)
- ***?Do all words carry the same importance?***

TF-IDF

- Term Frequency – Inverse Document Frequency
- **TF** = **how often** a term appears in a document
 - the more frequently a word appears in a single document, the more important that term is to the document
- **IDF** = how **unique** a term is across documents
 - words that appear more frequently in the corpus give less insight into the topic or meaning of an individual document

TF-IDF II.

- Combined → weights words higher if they are **frequent in one document but rare across others**
- indicate how important a word is to each document in a collection of documents, or a corpus
- a higher TF-IDF score indicates a term is more important to the corresponding document

*TFIDF score for term i in document j = $TF(i,j) * IDF(i)$*

where

IDF = Inverse Document Frequency

TF = Term Frequency

$$TF(i,j) = \frac{\text{Term i frequency in document j}}{\text{Total words in document j}}$$

$$IDF(i) = \log_2 \left(\frac{\text{Total documents}}{\text{documents with term i}} \right)$$

and

t = Term

j = Document

source:

<https://mungingdata.wordpress.com/2017/11/25/episode-1-using-tf-idf-to-identify-the-signal-from-the-noise/>

TF-IDF

docs = ["A long time ago in a galaxy far, far away.",
"A planet in a different galaxy is far away."]

- "galaxy" appears in both → IDF is low → less important.
- "long" or "planet" appears only in one of the sentences → IDF is high → more important.

TF-IDF: Example

- example taken from:
<https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/>
- let's count TF-IDF for a term in 3 documents:

Document 1: "The cat sat on the mat."

Document 2: "The dog played in the park."

Document 3: "Cats and dogs are great pets."
- Let's focus on the word "cat"

- Step 1: Calculate Term Frequency (TF)

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Document 1: "The **cat** sat on the mat."

For Document 1:

The word "cat" appears 1 time.

The total number of terms in Document 1 is 6 ("the", "cat", "sat", "on", "the", "mat").

So, $TF(\text{cat}, \text{Document 1}) = 1/6$

Document 2: "The dog played in the park."

For Document 2:

The word "cat" does not appear.

So, $TF(cat, Document\ 2) = 0$.

Document 3: "**Cats** and dogs are great pets."

For Document 3:

The word "cat" appears 1 time.

The total number of terms in Document 3 is 6 ("cats", "and", "dogs", "are", "great", "pets").

So $TF(cat, Document\ 3) = 1/6$

- Step 2: Calculate Inverse Document Frequency (IDF)

$$\text{IDF}(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

Total number of documents in the corpus (D): 3

Number of documents containing the term "cat": 2 (Document 1 and Document 3).

$$\text{IDF}(\text{cat}, D) = \log 3/2 \approx 0.176$$

- Step 3: Calculate TF-IDF

$$\text{TF - IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

For Document 1: $\text{TF-IDF}(\text{cat}, \text{Document 1}, D) = 0.167 * 0.176 = 0.029$

For Document 2: $\text{TF-IDF}(\text{cat}, \text{Document 2}, D) = 0 * 0.176 = 0$

For Document 3: $\text{TF-IDF}(\text{cat}, \text{Document 3}, D) = 0.167 * 0.176 = 0.029$

TF-IDF in Python

```
docs = ["A long time ago in a galaxy far, far away.",  
        "A planet in a different galaxy is far away."]
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer(stop_words='english')
```

```
X_tfidf = tfidf.fit_transform(docs)
```

```
pd.DataFrame(X_tfidf.toarray(), columns=tfidf.get_feature_names_out())
```

term-document
matrix →

	ago	away	different	far	galaxy	long	planet	time
0	0.40698	0.289569	0.000000	0.579139	0.289569	0.40698	0.000000	0.40698
1	0.00000	0.379303	0.533098	0.379303	0.379303	0.00000	0.533098	0.00000

TF-IDF Review

- Term frequency-inverse document frequency, known as TF-IDF, is a numerical statistic used to indicate how important a word is to each document in a collection of documents
- TF-IDF consists of two components, term frequency and inverse document frequency
- term frequency is how often a word appears in a document
- inverse document frequency is a measure of how often a word appears across all documents of a corpus
- TF-IDF is calculated as the term frequency multiplied by the inverse document frequency

Source:

<https://www.codecademy.com/courses/intro-to-language-models-in-python/lessons/language-model-tf-idf/exercises>

BoW vs. TF-IDF

- **BoW** → frequency only. Shared/common words can overshadow distinctive ones.
- **TF-IDF** → adjusts weights so *rare-but-informative words stand out*.

TF-IDF in Applications

Ranking results in search engines

- TF-IDF scores help decide which documents are most relevant to a user query.

Text summarization

- Selects the most “important” sentences or keywords by weighting unique terms higher.

Spam filtering / document classification

- Words like “free”, “offer”, “win” may get high TF-IDF scores in spam messages, helping classifiers.

Keyword extraction

- Identify key terms that represent a document, useful for tagging or indexing.

c-TF-IDF

- c-TF-IDF stands for class-based TF-IDF
- It's a variation of regular TF-IDF used in BERTopic to find topic-specific keywords.

c-TF-IDF

- Standard TF-IDF computes word importance **per document**.
- In topic modeling, we **cluster multiple documents together** into a topic.
- We want words that **distinguish one topic from others**, not just frequent words in one single document.



Example:

- Topic 1: Star Wars sentences
- Topic 2: Planetary science sentences
- Words like „spaceship" should stand out in Topic 1; words like „supernova" in Topic 2.

c-TF-IDF: How it works (conceptually)

1. Merge all documents in a topic into a single “super-document”.
2. Compute **TF-IDF** for each term across topics instead of documents.
 - **TF (Term Frequency)**: frequency of the term in the topic’s super-document.
 - **IDF (Inverse Document Frequency)**: $\log$ of (number of topics $\div$ number of topics containing the term).

c-TF-IDF

c-TF-IDF

For a term x within class c :

$$W_{x,c} = \| \mathbf{tf}_{x,c} \| \times \log \left(1 + \frac{A}{f_x} \right)$$

$\mathbf{tf}_{x,c}$ = frequency of word x in class c

f_x = frequency of word x across all classes

A = average number of words per class

Source: https://maartengr.github.io/BERTopic/getting_started/ctfidf/ctfidf.html

c-TF-IDF: Intuition

- TF $\rightarrow$ how common the word is **within the topic**.
- IDF $\rightarrow$ reduces weight for words that appear across **many topics**.
- Outcome: words that are **representative of a topic** get high scores.



Simple analogy:

- Regular TF-IDF $\rightarrow$ “what words are important in this document?”
- c-TF-IDF $\rightarrow$ “what words are important for this **cluster of documents**?”

c-TF-IDF: Example

- Doc 1: "A long time ago in a galaxy far, far away."
- Doc 2: "A planet in a different galaxy is far away."
- Doc 3: "Pluto is not a planet anymore."

Merged per topic:

- Topic 1: "A long time ago in a galaxy far far away"
- Topic 2: "A planet in a different galaxy is far away. Pluto is not a planet anymore."

c-TF-IDF will highlight:

- Topic 1 → "long", "time", "ago"
- Topic 2 → "planet", "different", "Pluto"
- Shared words like "galaxy", "far", "away" get lower scores

Problems with TF-IDF

- it does not understand the semantics
- similar words ('cat' and 'kitty') are treated as two separate units
- homonyms ('apple' and 'Apple') are treated as identical (if lowercased)
- Solution: Semantics is captured by another vector representation: **semantic embeddings**

Revision

CountVectorizer

TF-IDF

Tokenization

Pre-Processing

c-TF-IDF

Bag-of-Words

Frequencies

1. Clean and standardize raw text to make it suitable for analysis.
2. Split text into smaller units (tokens) such as words or subwords.
3. Count how often each token appears in a document or corpus.
4. Represent text as a vector of token counts, ignoring word order.
5. A Python implementation of BoW, with parameters for stopwords, n-grams, binary counts, and min/max document frequency.
6. Weight tokens by frequency in a document and rarity across documents to highlight important words.
7. Compute TF-IDF across topic clusters to identify words that distinguish one topic from others.

Revision

- Preprocessing: Clean and standardize raw text to make it suitable for analysis.
- Tokenization: Split text into smaller units (tokens) such as words or subwords.
- Frequencies: Count how often each token appears in a document or corpus.
- Bag-of-Words (BoW): Represent text as a vector of token counts, ignoring word order.
- CountVectorizer: A Python implementation of BoW, with parameters for stopwords, n-grams, binary counts, and min/max document frequency.
- TF-IDF: Weight tokens by frequency in a document and rarity across documents to highlight important words.
- c-TF-IDF: Compute TF-IDF across topic clusters to identify words that distinguish one topic from others.

Take-Aways

- What is pre-processing and why do we do it?
- What are types and tokens?
- What are n-grams?
- What is Bag-of-Words? What is the difference between binary and frequency BoW?
- What does TF-IDF do?

Want to learn more?

- Vectorizers:
https://maartengr.github.io/BERTopic/getting_started/vectorizers/vectorizers.html
- c-TF-IDF:
https://maartengr.github.io/BERTopic/getting_started/ctfidf/ctfidf.html
- BERT Tokenization: <https://huggingface.co/learn/llm-course/chapter6/6>
- interactive course about BoW, TF-IDF and embeddings:
<https://www.codecademy.com/enrolled/courses/intro-to-language-models-in-python>
- TF-IDF:
<https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/>

Thank you

klara.venglarova@uni-graz.at

